Agentic RAG — Internal Notes

Detailed technical reference for the agenticrag.ipynb notebook. Not for public distribution.

Project Structure

```
Agentic Rag/
├── agenticrag.ipynb     # main notebook
├── documents/           # put your PDF, TXT, DOCX, PPTX, CSV, MD files here
├── faiss_index/         # auto-created on first run, reused on subsequent runs
├── .env                 # ANTHROPIC_API_KEY=... and TAVILY_API_KEY=...
└── INTERNAL_NOTES.md
```

Cell Guide

Cell 1 — Imports

Installs `tavily-python` and loads all libraries: LangGraph for the agent graph, LangChain for document loaders and vector store, Anthropic for the LLM, HuggingFace for local embeddings, `TavilySearchResults` for the web search fallback, and `ThreadPoolExecutor` from `concurrent.futures` for parallel juror execution.

Cell 2 — Environment

Loads the `.env` file to read `ANTHROPIC_API_KEY` without hardcoding it.

Cell 3 — LLM and Embeddings

Sets `ANTHROPIC_API_KEY` and `TAVILY_API_KEY` from the environment, then initialises two models:

- `llm` — Claude Sonnet via Anthropic API. Used for all reasoning (decide, query reformulation, web search gating, generation).

- `embeddings` — `all-mpnet-base-v2` from HuggingFace, runs locally. Converts text to vectors for similarity search.

Cell 4 — State Definition

Defines `AgentState` — see full breakdown below.

Cell 5 — Vectorstore and Retriever

Loads documents, chunks them, builds a FAISS index, and saves it to disk. See full breakdown below.

Cell 6 — Agent Function

Defines all node functions, routing functions, builds the LangGraph graph, and exposes `run_agent()`. See full breakdown below.

Cell 7 — Testing

Test cells for single questions and multi-turn conversation.

State Definition (Cell 4)

`AgentState` is a typed dictionary that acts as the **shared memory** passed between every node in the graph. Every node reads from it and writes back to it.

```
class AgentState(TypedDict):
    query: str
    retrieved_docs: list[Document]
    answer: str
    route_decision: str
    retrieval_count: int
    chat_history: list[str]
    confidence_score: float
    escalation_flags: list[str]
```

Field	Type	What it holds
`query`	`str`	The current question. The `query` node can overwrite this with a reformulated version.
`retrieved_docs`	`list[Document]`	Chunks from local docs or web results. Each `Document` has `.page_content` and `.metadata["source"]` (filename or URL).
`answer`	`str`	The final answer produced by `jury_generate`. Empty string until generation runs.
`route_decision`	`str`	Set by `decide`. One of `"local"` (search local docs), `"web"` (search the web), or `"general"` (answer from LLM knowledge).
`retrieval_count`	`int`	Incremented every time `retrieve` runs. Enforces the 2-attempt cap to prevent infinite loops.
`chat_history`	`list[str]`	Running log of past turns: `["User: ...", "Assistant: ...", ...]`. Injected into the generate prompt for multi-turn context.
`confidence_score`	`float`	Set by the judge inside `jury_generate`. 0.0–1.0 reflecting how well the sources support the answer.
`escalation_flags`	`list[str]`	Populated by `escalation_gate`. Empty if clean, otherwise contains reasons why the answer was flagged.

`TypedDict` is used because LangGraph validates the exact shape of state at compile time.

Vectorstore and Retriever (Cell 5)

This cell is responsible for turning raw files into a searchable vector index and exposing a retriever the agent can query.

LOADERS

```
LOADERS = {
    ".pdf":  PyPDFLoader,
    ".txt":  TextLoader,
    ".docx": Docx2txtLoader,
    ".doc":  Docx2txtLoader,
    ".csv":  CSVLoader,
    ".md":   UnstructuredMarkdownLoader,
    ".pptx": UnstructuredPowerPointLoader,
}
```

A mapping from file extension to LangChain loader class. Adding support for a new file type only requires adding one entry here — the rest of the pipeline picks it up automatically.

load_document

```
def load_document(file_path: str) -> list[Document]:
    ext = os.path.splitext(file_path)[1].lower()
    loader_cls = LOADERS.get(ext)
    ...
    docs = loader_cls(file_path).load()
    for doc in docs:
        doc.metadata["source"] = os.path.basename(file_path)
    return docs
```

- Looks up the correct loader class from `LOADERS` by extension
- Unsupported extensions are skipped with a `[SKIP]` message — no crash
- Errors (corrupted files, missing dependencies) are caught and logged as `[ERROR]` — loading continues with the remaining files
- Stamps every `Document` with `metadata["source"] = filename` so the agent can cite the file by name when generating answers

`build_vectorstore`

```
def build_vectorstore(folder_path, chunk_size=500, chunk_overlap=100):
    # 1. collect all supported files recursively
    # 2. load each file → list[Document]
    # 3. split into chunks
    # 4. embed and store in FAISS
```

Four-step pipeline:

1. **Collect** — `glob` recursively finds every file with a supported extension inside `folder_path`
2. **Load** — calls `load_document` on each file; accumulates all pages across all files into one flat list
3. **Split** — `RecursiveCharacterTextSplitter` cuts documents into 500-character chunks with 100-character overlap. Overlap ensures that a sentence at the boundary of two chunks isn't lost. Separators are tried in order (`\n\n` , `\n` , `.` , `  ` , `""`) so splits fall on natural boundaries when possible
4. **Embed + index** — `FAISS.from_documents` embeds every chunk using the local `all-mpnet-base-v2` model and stores the resulting vectors in a FAISS index for fast similarity search

Index Persistence

```
FAISS_INDEX_PATH = "faiss_index"

if os.path.exists(FAISS_INDEX_PATH):
    vectorstore = FAISS.load_local(FAISS_INDEX_PATH, embeddings,
                                   allow_dangerous_deserialization=True)
else:
    vectorstore = build_vectorstore("documents")
    vectorstore.save_local(FAISS_INDEX_PATH)
```

On every run the cell checks for a saved index first:

- **Index exists** → load from disk in seconds; no re-embedding needed
- **No index** → build from scratch (can take a minute depending on document size), then save to `faiss_index/` for next time

`allow_dangerous_deserialization=True` is required by LangChain when loading a FAISS index from a pickle file. It is safe here because you control the `faiss_index/` folder.

Retriever

```
retriever = vectorstore.as_retriever(
    search_type="mmr",
    search_kwargs={"k": 4, "fetch_k": 20}
)
```

Wraps the vectorstore as a LangChain retriever using **MMR (Maximal Marginal Relevance)**:

- `fetch_k=20` — retrieve the 20 most similar chunks by raw cosine similarity
- `k=4` — from those 20, pick the 4 that are both relevant **and** diverse (MMR re-ranks to avoid returning near-duplicate chunks)

This means the agent always gets 4 chunks that cover different parts of the topic rather than 4 nearly identical excerpts from the same paragraph.

Agent Function (Cell 6)

The agent is a LangGraph `StateGraph` — a directed graph where each node is a Python function that transforms the state, and edges define the flow.

`_available_sources`

```
_indexed_sources = {
    doc.metadata.get("source", "unknown")
    for doc in vectorstore.docstore._dict.values()
}
_folder_sources = {
    os.path.basename(f)
    for ext in LOADERS
    for f in glob.glob(os.path.join("documents", f"**/*{ext}"), recursive=True)
}
_available_sources = list(_indexed_sources)

_unindexed = _folder_sources - _indexed_sources
if _unindexed:
    print(f"WARNING: files in documents/ but NOT in index: {_unindexed}")
```

- `_indexed_sources` — filenames actually in the FAISS index (what the agent can search)
- `_folder_sources` — filenames currently in the `documents/` folder
- `_available_sources` — always set to the indexed sources only, so `decide` never routes to a file that can't be retrieved
- The warning fires at startup if the two sets don't match — meaning a file was added to `documents/` but the index hasn't been rebuilt yet. It's a `print()` only — the agent runs fine either way.

Node Functions

`decide`

```
prompt = f"""Classify the question into ONE of:
- 'local'  : relates to the listed documents
- 'web'    : factual but not in the documents (news, current stats, recent events)
- 'general': answerable from general knowledge (math, definitions, casual chat)
Answer ONLY 'local', 'web', or 'general'."""
state["route_decision"] = decision   # "local" | "web" | "general"
```

- The entry point — every question starts here
- Shows the LLM the exact list of indexed filenames to inform the `"local"` vs `"web"` decision

- Three-way classification: `"local"` → search docs, `"web"` → Tavily, `"general"` → LLM knowledge
- Resets `retrieval_count` to 0 for every new question
- Falls back to `"general"` if the LLM returns anything unexpected

`retrieve`

```
docs = retriever.invoke(state["query"])
state["retrieved_docs"] = docs
state["retrieval_count"] += 1
```

- Runs MMR search against the FAISS vectorstore (returns 4 diverse, relevant chunks)
- Stores chunks in `retrieved_docs`
- Increments `retrieval_count` so the router can enforce the retry cap

`query`

```
state["query"] = llm.invoke(reformulation_prompt).content.strip()
```

- Called when retrieved docs are judged not relevant enough
- Sends the original question + poor results to the LLM and asks for a better, more specific question
- Overwrites `state["query"]` — the next `retrieve` call uses this improved version

`jury_generate`

Replaces the old single `generate` node. Runs three specialist jurors **concurrently** using `ThreadPoolExecutor(max_workers=3)`, then a judge that synthesises their verdicts.

Juror 1 — Citation Juror
Answers strictly from explicit source statements only. Writes "NOT IN SOURCES" for anything not directly stated. Forces citations for every fact.

Juror 2 — Synthesis Juror
Connects the dots across all retrieved chunks. Draws reasonable inferences where chunks support each other. Aims for the most complete answer.

Juror 3 — Gap Juror
Explicitly identifies what the context fails to answer. Surfaces missing information before providing what it can. Keeps the user aware of blind spots.

Parallel execution

All three jurors are submitted to a `ThreadPoolExecutor` simultaneously. Since they each only need `query + retrieved_docs` (no dependency on each other), they run in true concurrency. The executor blocks until all three complete, then the results are handed to the judge. This reduces the jury phase from ~3 serial LLM round-trips to ~1 parallel round-trip.

```
with ThreadPoolExecutor(max_workers=3) as executor:
    f1 = executor.submit(lambda: llm.invoke(juror1_prompt).content.strip())
    f2 = executor.submit(lambda: llm.invoke(juror2_prompt).content.strip())
    f3 = executor.submit(lambda: llm.invoke(juror3_prompt).content.strip())
    verdict1 = f1.result()
    verdict2 = f2.result()
    verdict3 = f3.result()
```

Judge LLM

Receives all three verdicts and:

1. Synthesises a single final answer, resolving disagreements
2. Preserves citations from the jurors
3. Assesses whether jurors significantly disagreed (YES/NO)
4. Assigns a `confidence_score` 0.0–1.0 based on source support

```
JUROR_DISAGREED: YES or NO
CONFIDENCE: 0.0 to 1.0
ANSWER:
[synthesised answer]
```

The response is parsed line by line. If parsing fails, the raw judge response is used as the answer and confidence defaults to 0.7.

- Appends this turn to `chat_history` for future questions
- Sets `escalation_flags = ["juror disagreement"]` if judge detected disagreement

`escalation_gate`

```
flags = []
if confidence_score < CONFIDENCE_THRESHOLD:    flags.append("low confidence")
if not retrieved_docs:                         flags.append("general knowledge only")
if retrieved_docs and "[" not in answer:       flags.append("no citations despite sources")
```

- Runs after every `jury_generate` call without exception
- `CONFIDENCE_THRESHOLD = 0.6` — adjustable constant at the top of Cell 6
- Merges any juror disagreement flag set by `jury_generate` with its own checks
- Prints `[escalation] FLAGGED: ...` if any flag fires, `[escalation] clear` otherwise
- Does not block or reroute — always proceeds to END; flags are surfaced in `run_agent`

`web_search`

```
tool = TavilySearchResults(max_results=4)
results = tool.invoke(state["query"])
state["retrieved_docs"] = [
    Document(page_content=r["content"], metadata={"source": r["url"]})
    for r in results
]
```

- Reached in two ways: directly from `decide` when `route_decision="web"` , or as a fallback from `route_after_retrieve` when local retrieval is exhausted
- No gating LLM call — the decision to search was already made upstream
- Calls Tavily, converts each result into a `Document` where `page_content` is the snippet and `metadata["source"]` is the URL
- `jury_generate` then cites those URLs exactly like it cites filenames
- Always connects to `jury_generate` via a fixed edge — no branching after this node

Routing Functions

Called between nodes — not nodes themselves. Read state and return a string key.

`route_decide`

```
mapping = {"local": "retrieve", "web": "web_search", "general": "jury_generate"}
return mapping[state["route_decision"]]
```

Maps the three-way decision from `decide` to the correct next node — no LLM call, pure state read. General knowledge questions skip straight to `jury_generate` (jurors run with GK-aware prompts, no context block).

`route_after_retrieve`

```
if state["retrieval_count"] >= MAX_RETRIEVAL_ATTEMPTS:
    return "web_search"   # local retrieval exhausted → try web
# ask LLM if top doc is relevant
return "jury_generate" if relevant else "query"
```

- Hard stops local retrieval at 2 attempts — routes to `web_search`
- Otherwise asks the LLM if the top retrieved chunk answers the question
- Returns `"query"` to trigger reformulation and retry if not relevant
- Returns `"jury_generate"` (not `"generate"`) when docs are relevant

Constants

```
MAX_RETRIEVAL_ATTEMPTS = 2
CONFIDENCE_THRESHOLD   = 0.6
```

`MAX_RETRIEVAL_ATTEMPTS` — caps how many times `retrieve` can run per question. If the count reaches 2, `route_after_retrieve` routes to `web_search` regardless of relevance.

`CONFIDENCE_THRESHOLD` — the minimum acceptable confidence score from the judge. Any answer scoring below this triggers the "low confidence" escalation flag. Both constants are at the top of Cell 6 for easy tuning.

Building the Graph

```
graph = StateGraph(AgentState)

graph.add_node("decide",          decide)
graph.add_node("retrieve",        retrieve)
graph.add_node("query",           query)
graph.add_node("web_search",      web_search)
graph.add_node("jury_generate",   jury_generate)
graph.add_node("escalation_gate", escalation_gate)

graph.set_entry_point("decide")

graph.add_conditional_edges("decide", route_decide, {
    "retrieve":      "retrieve",       # route_decision="local"
    "web_search":    "web_search",     # route_decision="web"
    "jury_generate": "jury_generate",  # route_decision="general"
})
graph.add_conditional_edges("retrieve", route_after_retrieve, {
    "jury_generate": "jury_generate",
    "query":         "query",
    "web_search":    "web_search",
})
graph.add_edge("query",           "retrieve")        # fixed
graph.add_edge("web_search",      "jury_generate")   # fixed
graph.add_edge("jury_generate",   "escalation_gate") # fixed
graph.add_edge("escalation_gate", END)               # fixed

agent = graph.compile()
```

`add_edge` = always go there.

`add_conditional_edges` = routing function decides at runtime.

The graph is wrapped in `build_agent()` so it can be reconstructed cleanly if needed (e.g. in tests or after reloading the vectorstore). The compiled agent is stored in the module-level variable `agent`.

Runner

```
_SOURCE_LABELS = {"local": "Local Documents", "web": "Web Search", "general": "General Knowledge
_chat_history: list[str] = []

def run_agent(question: str, reset_history: bool = False) -> str:
    ...
    print(f"  Source     : {source}")
    print(f"  Confidence : {confidence:.0%}")
    if flags:
        print(f"  FLAGGED    : {', '.join(flags)}")
    return result["answer"]
```

`run_agent` is the only public interface to the agent. It:

1. Optionally clears conversation history (`reset_history=True`) to start fresh
2. Initialises the full `AgentState` dict — includes new `confidence_score: 0.0` and `escalation_flags: []` defaults
3. Passes a **copy** of `_chat_history` into the state so the graph can't mutate the global list mid-run
4. After the graph finishes, writes the updated history back to `_chat_history`
5. Prints a metadata block showing source, confidence %, and any escalation flags
6. Returns just the `answer` string

Example output:

```
============================================================
  Source     : Local Documents
  Confidence : 87%
============================================================
```

Or if flagged:

```
============================================================
  Source     : Web Search
  Confidence : 45%
  FLAGGED    : low confidence (45%), juror disagreement
============================================================
```

Full Flow

```
Every question starts at decide (3-way classification)
        |
        |  "local"          "web"           "general"
        ↓                    ↓                  ↓
    retrieve            web_search           generate
        |                    |                  |
  docs relevant?        Tavily results         |
     yes ──────────────────────────────────► generate
      |                                        |
      | no + attempts < 2                     END
      ↓
  query (reformulate)
      |
  retrieve (retry)
      |
  attempts >= 2
      ↓
  web_search ──────────────────────────────► generate
                 Tavily results                |
                                               END
```

How to Test

1. Run all cells top to bottom

You should see:

```
Available documents: ['LOGISTIC_VS_LINEAR.pdf', ...]
Agent compiled.
```

2. Single question

```
answer = run_agent("What is the difference between logistic and linear regression?")
print(answer)
```

Watch the `[decide]`, `[retrieve]`, `[route_after_retrieve]`, `[web_search]`, `[generate]` logs to see the exact path taken.

3. Multi-turn conversation

```
_chat_history.clear()
print(run_agent("What is logistic regression used for?"))
print("---")
print(run_agent("Can you give me an example of that?"))
```

The second question carries context from the first automatically.

4. View graph structure

```
from IPython.display import Image, display
display(Image(agent.get_graph().draw_mermaid_png()))
```

Adding New Documents

1. Drop files into `documents/` (supported: `.pdf`, `.txt`, `.docx`, `.pptx`, `.csv`, `.md`)
2. Delete the `faiss_index/` folder
3. Restart kernel and run all cells — index rebuilds automatically

If you forget step 2, the agent will still run but will print a warning at startup:

```
WARNING: files in documents/ but NOT in index: {'new_file.pdf'}
Delete faiss_index/ and restart to include them.
```

The agent won't crash — it just won't search the new file until you rebuild.